OHL Goalkeeper Progression Model

Data Pipeline Documentation

UCLL Project Week — March 2026

Prepared for: OH Leuven Data & IT Department

Central question: Given a goalkeeper's performance in a lower league, can we predict whether he will succeed at a higher level?

Overview

This document describes the complete data pipeline implemented in the Jupyter notebook gk_analysis.ipynb for the first three steps of the goalkeeper progression model:

Step	Name	Output
1	Load Data	Match-folder index, loader functions, definition maps
2	Feature Engineering	Per-keeper feature vectors, binary targets, exploratory plots
3	Feature Selection	Curated GK feature set, MI ranking, gk_features_ready.csv

Each step is described below with the rationale behind design decisions, known limitations, and the issues encountered and resolved during development.

Step 1: Load Data

1.1 Input files

The pipeline reads from four sources at the root of the data folder:

File	Purpose
gk_dataset_final.csv	Labels and match references for all 693 goalkeepers
player_score_definitions.json	Maps score ID integers to human-readable score names (134 scores)
player_kpi_definitions.json	Maps KPI ID integers to names (1,458 KPIs)
event_kpi_definitions.json	Maps event KPI IDs to names (103 event KPIs, optional)

Definition files are loaded into Python dictionaries keyed by integer ID for O(1) lookup when renaming columns later:

```
score_defs = {d["id"]: d["name"] for d in json.load(f).get("data", [])}
```

1.2 Match-folder index

Match data is stored in a two-level folder structure: competitions/<competition-season>/<match-folder>/. The match folders referenced in the origin_match_dirs column of the CSV are bare directory names (e.g. 20230401 Club A - Club B) without any competition-season prefix.

A naive approach would scan all 40+ competition-season folders on every lookup. Instead, we build a single dictionary at startup that maps every match folder name to its full path:

```
match_index: dict[str, Path] = {}
for cs_dir in COMP_DIR.iterdir():
    for match_dir in cs_dir.iterdir():
        match_index[match_dir.name] = match_dir
```

This indexes ~21,000 match folders once, reducing per-keeper lookups from O(n×competitions) to **O(1)**. Without this, loading all 693 keepers would be prohibitively slow on a 24 GB dataset.

1.3 Loader functions

Two loader functions were implemented, corresponding to the two main data levels described in the documentation:

Function	Data source	Level	Use case
load_keeper_scores()	player_scores.json	Level 1	Fast loading, 134 composite scores per match
load_keeper_kpis()	player_kpis.json	Level 2	Deeper analysis, up to 1,458 raw KPIs per match

Filters applied in both loaders

- **Playing time filter:** matchShare >= 0.5
- **Position filter:** position == "GOALKEEPER"
- **Training data scope:** Only origin_match_dirs are loaded (data from BEFORE the transfer)

The position cross-reference problem

player_scores.json does not contain a position field. It is therefore impossible to determine whether a player entry in that file is a goalkeeper or an outfield player from the scores file alone.

The solution implemented in load_keeper_scores() is to open player_kpis.json for the same match first, confirm that the target player ID has position == "GOALKEEPER" and matchShare >= 0.5, and only then read that player's scores from player_scores.json. This adds one extra file read per match but ensures position correctness.

> **Note:** This cross-reference is not needed for load_keeper_kpis() because player_kpis.json contains the position field directly.

1.4 Minimum matches filter

Keepers with fewer than 3 qualifying matches are excluded before feature engineering. This threshold reflects the project briefing guidance that very low playing time produces unreliable scores.

> **Warning:** Setting this threshold too high (e.g. 10+) risks losing the BENCH class (only 31 keepers), which is already the smallest category. The default of 3 is conservative; consider raising it to 5 for modelling if class balance allows.

Step 2: Feature Engineering

2.1 Aggregation strategy

Each keeper may have played between 1 and 50+ qualifying matches at their origin club. The raw data is a list of per-match score dictionaries. These are aggregated into a single feature vector per keeper using two statistics:

Statistic	Column prefix	Rationale
Mean	mean_	Primary feature: average performance level across all matches
Standard deviation	std_	Reliability indicator: high std = inconsistent performer

The mean is the primary signal. The standard deviation is included as an auxiliary feature because match-to-match consistency may itself be a predictor of progression, and because the project briefing specifically flags high variance as a concern when scouting on data.

2.2 Column renaming

Raw column names from the loader are of the form score_164, score_192, etc. These are renamed to mean_GK_PREVENTED_GOALS_TOTAL_POSTSHOT_XG, mean_GK_SUCCESSFUL_LAUNCHES_PERCENT, etc. using the score_defs dictionary loaded in Step 1. Human-readable names make exploratory analysis and feature importance plots interpretable without constantly cross-referencing the definition file.

2.3 Binary target variables

Two binary targets are derived from the status column:

Target	Positive class (1)	Negative class (0)	Modelling question
y_plays	PLAYS	BENCH + STAYED + DROPPED	Will this keeper break through and play regularly?

Target	Positive class (1)	Negative class (0)	Modelling question
y_up	PLAYS + BENCH	STAYED + DROPPED	Will this keeper earn any higher-level exposure?

y_plays is the primary target. It corresponds directly to the central project question. y_up is a secondary target that may be easier to predict and can serve as a sanity check.

2.4 Class imbalance

The dataset is heavily imbalanced. The distribution after the minimum-matches filter is approximately:

Status	Approx. count	y_plays	y_up
STAYED	~380	0	0
DROPPED	~110	0	0
PLAYS	~85	1	1
BENCH	~25	0	1

> **Warning:** BENCH has only ~25 samples after filtering. Do not drop this class — it is a meaningful negative outcome (transfer happened but keeper failed to play). Handle imbalance in Step 4 using class_weight='balanced' or SMOTE, not by removing classes.

2.5 Exploratory box plots

Box plots are generated for two groups of scores, split by status category (PLAYS / BENCH / DROPPED / STAYED):

- GK-specific scores (IDs 164–192): shot stopping, aerial play, sweeper activity, distribution
- General scores with good coverage (IDs 0, 9, 10, 17, 52, 55, 81, 101, 163, 232)

These plots are saved as gk_scores_by_status.png and general_scores_by_status.png. The purpose is to visually confirm which metrics show separation between the PLAYS group and the others before any modelling is performed, consistent with the project briefing guidance to explore before modelling.

Step 3: Feature Selection

3.1 The problem with blind mutual information ranking

The initial implementation applied mutual information (MI) directly to all 134 score features. This produced a nonsensical top-20 list dominated by outfield-specific scores such as OPEN_GOAL_SHOT_SCORE, HOLD_UP_PLAY_SCORE, and FREE_KICK_SCORE.

The root cause is that these scores are near-zero for every goalkeeper. However, the variance in those near-zeros — driven by how many matches each keeper appeared in, not by goalkeeper quality — creates spurious MI signal. A keeper appearing in 30 matches will have a more stable near-zero than a keeper appearing in 3 matches, and MI detects this variance pattern rather than any meaningful relationship with the target.

Additionally, the first iteration used a global median fill for all NaN values before running MI. For scores with very low coverage (e.g. GK_PREVENTED_GOALS_* only available from the 2025-2026 season), filling with the column median produces a near-constant column, which MI correctly scores near zero — but this masks the score's potential value for the subset of keepers where it is available.

3.2 Curated GK feature set

The fix is to pre-filter to a manually curated list of GK-relevant score IDs before running MI. This list is derived directly from the documentation's GK-specific player scores section:

Category	Score IDs	What it measures
Shot stopping (post-shot xG)	164, 166, 167, 168, 169, 170, 171	Goals prevented relative to post-shot xG, by shot type
Shot stopping (pre-shot xG)	184, 186	Goals prevented relative to pre-shot xG
Aerial play	190, 191	Percentage of high balls caught or punched
Sweeper activity	189	Defensive touches outside own box
Distribution	192, 81, 52, 55, 163, 232	Launch success, goal kick quality, pass scores, accuracy
Overall	0, 17, 101	Overall Impect score, defensive Impect, total touches

> **Note:** Post-shot xG scores (IDs 164, 166–171) are conceptually stronger than pre-shot xG scores (IDs 184, 186) because they account for where the shot is heading in the goal, not just where it came from. The project briefing explicitly recommends prioritising post-shot xG. These scores are also the most likely to have low coverage due to the 2025-2026 availability restriction.

3.3 Missing-value handling

Two separate strategies are applied depending on why a value is missing:

Scenario	Strategy	Rationale
Score missing for a keeper because they played in an older season (GK_PREVENTED_GOALS_*)	Group-median imputation by status, then global median	A PLAYS keeper's missing shot-stopping score is better estimated from other PLAYS keepers than from the full population
Score missing for a keeper because they had too few matches (std_ columns)	Global median fill after the min-matches filter	Acceptable because min-matches filter already removes the most unreliable keepers
Entire score column missing for >40% of keepers	Column dropped before MI ranking	A column that is empty for the majority of keepers carries no usable signal

> **Warning:** Do NOT drop keepers who have NaN on GK_PREVENTED_GOALS_* scores. These are keepers from older seasons where Impect did not yet track this metric. They are not low-quality keepers — they simply pre-date the data collection. Dropping them would gut the PLAYS class and introduce a severe recency bias.

A binary has_ flag column is added alongside each imputed score column (e.g. has_GK_PREVENTED_GOALS_TOTAL_POSTSHOT_XG). This allows the model to distinguish keepers who genuinely scored low from keepers whose value was imputed, and data availability itself may correlate with recency.

3.4 Mutual information ranking

MI is computed against y_plays using sklearn.feature_selection.mutual_info_classif after:

- Restricting to the curated GK feature list plus three context features (age, origin_median, n_matches_loaded)
- Applying the missing-value filter (drop columns >40% missing)
- Filling remaining NaNs with group-median imputation

The resulting rankings from the first clean run showed the following top features:

Rank	Feature	MI score	Interpretation
1	GK_CAUGHT_AND_PUNCHED_HIGH_BALLS_PERCENT	0.065	Aerial dominance is the strongest individual signal
2	origin_median	0.063	League strength of origin is

Rank	Feature	MI score	Interpretation
			highly predictive
3	GK_PREVENTED_GOALS_POST_SHOT_XG_BY_ACTION_HEADER	0.040	Shot stopping on headers
4	GK_DEFENSIVE_TOUCHES_OUTSIDE_OWN_BOX	0.034	Sweeper activity
5	RATIO_PASSING_ACCURACY	0.023	Distribution quality

> **Note:** origin_median ranking second is expected and meaningful: keepers from stronger lower leagues face better opponents and may genuinely be higher quality. However, it also acts as a confounder. Consider running MI and modelling both with and without origin_median to isolate whether performance metrics alone carry predictive power.

3.5 Correlation deduplication

After MI ranking, pairs of features with absolute Pearson correlation above 0.85 are identified. From each such pair, the feature with the lower MI score is dropped. This reduces multicollinearity before modelling without discarding any unique signal.

Pairs most likely to be highly correlated in this feature set are:

- GK_PREVENTED_GOALS_TOTAL_POSTSHOT_XG and GK_PREVENTED_GOALS_TOTAL_POSTSHOT_XG_PERCENT (same underlying metric, different normalisation)
- GK_PREVENTED_GOALS_TOTAL_SHOT_XG and GK_PREVENTED_GOALS_TOTAL_SHOT_XG_PERCENT
- IMPECT_SCORE_PACKING and DEFENSIVE_IMPECT_SCORE_PACKING (overall score subsumes defensive score)

3.6 Output

The final output of Step 3 is gk_features_ready.csv, containing:

- playerId, name, status, direction (metadata, not used as model features)
- y_plays, y_up (binary targets)
- All selected mean_ features after the missing-value filter and correlation deduplication
- has_ binary flags for GK_PREVENTED_GOALS_* scores

This file is the direct input to Step 4 (modelling) and does not need any further preprocessing before fitting a classifier.

Known Limitations and Open Issues

Issue	Impact	Recommended action
GK_PREVENTED_GOALS_* available only from 2025-2026 for some competitions	Most keepers have NaN for the conceptually strongest shot-stopping features	Use group-median imputation + has_ flag; consider a separate model trained only on keepers with this data
origin_median is a league-level feature, not a keeper-level feature	Two keepers from the same league always have the same origin_median regardless of their actual performance	Use as a covariate or normalisation factor, not as a primary predictor
Age is calculated as of March 2026, not at time of transfer	Younger keepers who transferred early will have artificially high age values	Consider computing age_at_transfer from birthdate and origin_season if precise age effects are important
BENCH class has only ~25–31 samples	High risk of overfitting on this class; metrics on BENCH will have wide confidence intervals	Apply class_weight='balanced' in all classifiers; report confidence intervals on per-class metrics
std_ features may be unreliable for keepers close to the 3-match minimum	Standard deviation from 3 matches is highly unstable	Consider raising the minimum-matches threshold or excluding std_ features from the final model

Next Steps: Step 4 — Modelling

The gk_features_ready.csv file produced by Step 3 is ready for classification. The recommended modelling sequence is:

1. **Logistic Regression baseline** — interpretable coefficients, quick to run, establishes a minimum bar
2. **Random Forest** — handles non-linearities, built-in feature importance, robust to correlated features
3. **XGBoost / LightGBM** — strongest predictive performance, supports class weights natively

Evaluation should use stratified 5-fold cross-validation and report precision, recall, F1, and AUC-ROC per class. Feature importance from the tree-based models should be compared against the MI ranking from Step 3 as a consistency check.

Step 4: Modelling

Step 4 trains three binary classifiers on the feature matrix produced by Step 3 (gk_features_ready.csv) to answer the central question: *Given a goalkeeper's performance in a lower league, will he become a regular starter if he moves up?* The target variable is y_plays (1 = PLAYS, 0 = BENCH + STAYED + DROPPED).

4.1 Problem Framing

The problem is framed as binary classification with y_plays as the primary target. PLAYS is the only category that represents genuine success at the higher level — a keeper who transfers but never plays (BENCH) has not proven himself. The secondary target y_up (PLAYS + BENCH vs STAYED + DROPPED) is used as a sanity check to verify the features capture transfer-worthiness more broadly.

The input matrix after Step 3 contains 218 keepers and 20 features. The class distribution is approximately:

Status	Approx. count	y_plays	y_up
STAYED	~130	0	0
DROPPED	~60	0	0
PLAYS	~85	1	1
BENCH	~25	0	1

4.2 Cross-Validation Setup

All models are evaluated using **stratified 5-fold cross-validation**. Stratification ensures each fold preserves the same class ratio as the full dataset, which is critical given the imbalance between PLAYS (~39%) and the rest. Five metrics are recorded per fold:

- **AUC-ROC** — area under the ROC curve. Measures the model's ability to rank PLAYS keepers above non-PLAYS keepers regardless of threshold. A score of 0.5 equals random guessing; 1.0 is perfect. This is the primary evaluation metric.
- **Average Precision (AP)** — area under the Precision-Recall curve. More informative than AUC-ROC under class imbalance because it focuses specifically on performance on the minority (PLAYS) class.
- **F1 score** — harmonic mean of precision and recall at the default 0.5 threshold. Useful for comparing models at a fixed operating point.
- **Precision** — of all keepers the model flags as PLAYS, what fraction actually are? Relevant if OHL wants a short, high-confidence shortlist.
- **Recall** — of all actual PLAYS keepers, what fraction does the model catch? Relevant if OHL wants to miss as few genuine talents as possible.

> **Warning:** Raw accuracy is not reported. A model that always predicts "not PLAYS" would achieve ~61% accuracy on this dataset while being completely useless for scouting. AUC-ROC and AP are imbalance-resistant and are the correct metrics here.

4.3 Class Imbalance Handling

All three models use built-in class balancing so the minority class (PLAYS) is not drowned out by the majority classes during training. The mechanism differs per model:

Model	Parameter	Effect
Logistic Regression	class_weight='balanced'	Inversely weights each sample by its class frequency during training
Random Forest	class_weight='balanced'	Applied per bootstrap sample within each tree
XGBoost	scale_pos_weight = neg / pos	Scales the gradient of positive samples upward; equivalent to balanced weighting

4.4 The Three Models

Model 1 — Logistic Regression (baseline)

Logistic Regression is the starting point. Features are standardised inside a sklearn Pipeline (StandardScaler → LogisticRegression) so that coefficient magnitudes are directly comparable across features. L2 regularisation (default C=1.0) guards against overfitting on the small dataset. The primary purpose of this model is to establish a performance baseline and to produce interpretable coefficients: a positive coefficient on a feature means higher values push the prediction toward PLAYS.

Model 2 — Random Forest

Random Forest trains 500 decision trees on bootstrap samples of the data, each using a random subset of features at every split (max_features='sqrt'). The key hyperparameter choice for this dataset is min_samples_leaf=3, which prevents trees from growing splits that are supported by fewer than 3 samples — a necessary regularisation given the dataset has only 218 rows. Predicted probabilities are the average across all trees. This model also enables permutation importance, which measures how much the AUC-ROC drops when each feature's values are randomly shuffled across 30 repeats.

Model 3 — XGBoost

XGBoost builds trees sequentially, each one correcting the residual errors of the previous. The configuration uses shallow trees (max_depth=3) and a low learning rate (0.05) with 300 estimators, which together prevent overfitting on a small dataset by keeping individual trees weak and letting the ensemble improve gradually. Subsampling of rows (subsample=0.8) and columns (colsample_bytree=0.8) adds further regularisation. XGBoost is expected to produce the strongest predictive performance of the three models and is used for the threshold analysis and secondary target evaluation.

4.5 Evaluation Outputs

ROC and Precision-Recall curves (cell 4g)

Both curves are plotted using **out-of-fold probability estimates** obtained via cross_val_predict(method='predict_proba'). This means every keeper's probability score was generated by a model that had never seen that keeper during training, making the curves honest estimates of generalisation performance rather than in-sample fits. The PR curve also plots the random baseline (a horizontal line at the class prevalence ~0.39) so the degree of improvement over chance is immediately visible.

Confusion matrices (cell 4h)

Confusion matrices show the raw error breakdown at the default 0.5 threshold. Four cells are reported: true positives (PLAYS correctly identified), false positives (non-PLAYS wrongly flagged), false negatives (PLAYS missed), and true negatives. A full sklearn classification_report is printed for XGBoost, showing per-class precision, recall, and F1 with three decimal places.

Feature importance (cell 4i)

Two complementary importance methods are shown side by side. XGBoost gain importance measures how much each feature reduces the training loss when used in a split, averaged across all trees. Permutation importance on the Random Forest measures how much the out-of-sample AUC-ROC drops when each feature's values are randomly shuffled (30 repeats). These two methods answer different questions: gain importance reflects what the model learned to use; permutation importance reflects what actually helps on unseen data. Features that rank highly on both are the most trustworthy signals. The same orange/blue colour coding from Step 3 is used (orange = GK-specific scores, blue = general/context features).

4.6 Threshold Analysis (cell 4j)

The default classification threshold of 0.5 is rarely optimal under class imbalance. Cell 4j sweeps the threshold from 0.05 to 0.95, computing precision, recall, and F1 at each point using out-of-fold XGBoost probabilities. The threshold that maximises F1 is marked on the plot and used to print a final classification report.

The choice of operating threshold depends on OHL's scouting priorities:

Priority	Threshold choice	Trade-off
High recall	Lower threshold (e.g. 0.3)	Flag more keepers for review; miss fewer real talents but review more false positives
High precision	Higher threshold (e.g. 0.6)	Shorter, higher-confidence shortlist; risks missing some real talents
Balanced	Best-F1 threshold (computed automatically)	Optimal balance between precision and recall on this dataset

4.7 Secondary Target — y_up (cell 4k)

XGBoost is re-run on the secondary target y_up (positive class = PLAYS + BENCH, ~110 keepers; negative class = STAYED + DROPPED, ~108 keepers). This is a more balanced and easier problem. Two outcomes are informative:

- If y_up AUC is substantially higher than y_plays AUC, the features capture transfer-worthiness well but struggle to distinguish regular starters from bench-warmers. This points toward a data limitation rather than a modelling failure.
- If y_up and y_plays AUC are similar, the features have roughly equal discriminative power for both definitions of success.

4.8 Known Issue — Python 3.10 f-string Syntax

The summarise_cv() helper function in cell 4b uses f-strings with nested expressions. Python 3.10 does not support nested quotes inside f-strings (this was only allowed from Python 3.12 onwards). The fix is to assign the repeated string to a variable before the f-string:

```
sep = '=' * 55
print(f'\n{sep}')
```

This applies to all f-strings in the notebook that nest string expressions. Moving repeated string constructions to a named variable before the f-string is the correct pattern for Python 3.10.

4.9 Known Limitations

Issue	Impact	Recommended action
Only 218 keepers after min-matches filter	High variance in CV metrics; wide confidence intervals	Always report mean ± std across folds; do not overinterpret small differences between models
age MI = 0	Age as of March 2026 does not separate status categories; may be included but carries no predictive signal alone	Consider replacing with age_at_transfer derived from birthdate and origin_season
No hyperparameter tuning in Step 4	Default or manually chosen hyperparameters may not be optimal	Use nested CV with GridSearchCV or Optuna in Step 5 if performance warrants it

Step 5: Validation

Step 5 honestly assesses how well the XGBoost model from Step 4 generalises to unseen data. Rather than reporting a single headline number, validation examines the model from five angles: fold-by-fold stability, per-class performance, a qualitative error analysis, the shape of the predicted probability distribution, and a train-versus-validation gap check for overfitting. All evaluations use **out-of-fold predictions** obtained via cross_val_predict — meaning every keeper's score was produced by a model that had never seen that keeper during training.

5.1 Per-Fold Stability (cell 5a)

Step 4 reported mean ± std across the five folds. Cell 5a unpacks that into individual fold results, printing AUC-ROC, Average Precision, F1, Precision, and Recall for each fold alongside the number of keepers and PLAYS instances in each test set.

The key question is whether performance is **consistent across folds** or whether one fold is dominating the mean. A standard deviation above 0.10 on AUC-ROC signals that the dataset is too small to produce reliable estimates, and conclusions should be stated with explicit uncertainty. This is a direct consequence of having only 218 keepers after the minimum-matches filter — each fold contains roughly 43 keepers, of whom only ~17 are PLAYS.

> **Note:** Stratified splitting ensures each fold preserves the same PLAYS ratio as the full dataset. Without stratification, one fold could end up with very few PLAYS instances, making its metrics meaningless.

5.2 Per-Class Classification Report (cell 5b)

The most important validation output. sklearn's classification_report breaks down precision, recall, and F1 separately for the positive class (PLAYS) and the negative class (Other = BENCH + STAYED + DROPPED). Three additional numbers are printed:

Metric	What it tells you
AUC-ROC	Overall ranking ability. 0.5 = random, 1.0 = perfect. Primary metric.
Average Precision	Performance on PLAYS specifically. Random baseline = class prevalence (~0.39). Any value above this is genuine signal.
Precision (PLAYS)	Of all keepers the model flags as PLAYS, what fraction actually are? Relevant for shortlist quality.
Recall (PLAYS)	Of all actual PLAYS keepers, what fraction did the model catch? Relevant for not missing talents.
F1 (PLAYS)	Harmonic mean of precision and recall. Summary score at the 0.5 threshold.

The random baseline for Average Precision is printed alongside the model score so the improvement over chance is explicit. A model that simply predicts "PLAYS" for every keeper would achieve AP = 0.39 (the class prevalence). Any model result below this threshold has failed to learn anything useful.

5.3 Error Analysis (cell 5c)

Cell 5c prints two lists of individual keepers:

- **False Negatives** — actual PLAYS keepers the model missed, sorted by predicted probability. These are talents the scouting tool would have overlooked. Patterns in their profiles (origin league, number of matches, which features were low) point toward the model's blind spots.
- **False Positives** — non-PLAYS keepers the model wrongly flagged, sorted by predicted probability descending (highest-confidence wrong calls first). These would be wasted scouting effort. Checking their status (BENCH, STAYED, or DROPPED) shows whether the model confuses a specific category with PLAYS.

This qualitative step is as important as the quantitative metrics. A model that fails on a recognisable subgroup — for example, always missing keepers from the Regionalliga, or always flagging DROPPED keepers as PLAYS — reveals a systematic bias that the aggregate AUC number hides. The error list should be reviewed together with the domain expert (OHL's sporting staff) before the model is used for scouting decisions.

5.4 Predicted Probability Distribution (cell 5d)

Cell 5d plots a histogram of the model's predicted PLAYS probability for each of the four status categories (PLAYS, BENCH, DROPPED, STAYED) on the same axes. A well-calibrated, discriminating model should produce a clear visual separation: PLAYS keepers clustering toward 1.0, all others clustering toward 0.0.

Three patterns to look for in this plot:

Pattern	What it means	Action
PLAYS peaks near 1.0, others near 0.0	Good separation — model is confident and largely correct	Trust the scores for scouting
All distributions overlap heavily around 0.3–0.6	Model is uncertain — weak separation between classes	Use only the highest-probability keepers; report uncertainty to OHL
BENCH overlaps strongly with PLAYS	Model cannot distinguish starters from bench-warmers — features do not capture this difference	Compare y_plays vs y_up AUC from Step 4k; consider additional features

5.5 Overfitting Check (cell 5e)

Cell 5e computes AUC-ROC on the **training portion** of each fold and compares it to the validation AUC of the same fold. A large gap between train and validation AUC indicates that the model is memorising the training data rather than learning generalisable patterns — a serious risk with only 218 samples.

Train–Val gap	Interpretation	Recommended action
< 0.08	No strong signs of overfitting	Proceed with current configuration
0.08 – 0.15	Moderate overfitting — interpret results with caution	Increase min_samples_leaf in Random Forest; reduce max_depth or n_estimators in XGBoost

> 0.15	Likely overfitting — validation scores unreliable	Reduce model complexity significantly; consider Logistic Regression as the final model

> **Warning:** A near-perfect training AUC (above 0.95) combined with a mediocre validation AUC (below 0.70) is a strong indicator that XGBoost has memorised the 174 training samples rather than finding generalisable patterns. In that case the Logistic Regression baseline — which is harder to overfit due to its linear structure — may be the more honest and deployable model despite lower raw performance.

5.6 Interpreting Results for OHL

The validation step produces results that fall into one of three broad categories. Each has a different recommended conclusion for the OHL presentation:

Outcome	Indicator	Conclusion to report
Model works	AUC > 0.70, AP well above 0.39, stable folds, small train–val gap	These features are predictive of progression; the model can be used as a scouting filter with the recommended threshold
Weak signal	AUC 0.55–0.70, AP marginally above 0.39, high fold variance	Some features discriminate (report the top MI features from Step 3); full prediction is unreliable at this dataset size
No signal	AUC near 0.5, AP at or below 0.39, high overfitting gap	Impect player scores alone are insufficient; report this honestly and recommend collecting tracking data or a larger sample

> **Note:** The project briefing explicitly states that “an honest result that says we find no strong signal in this data is more valuable than a forced model with poor metrics.” Reporting a weak or null result clearly, with a well-reasoned explanation of why, is a valid and valued deliverable for OHL.